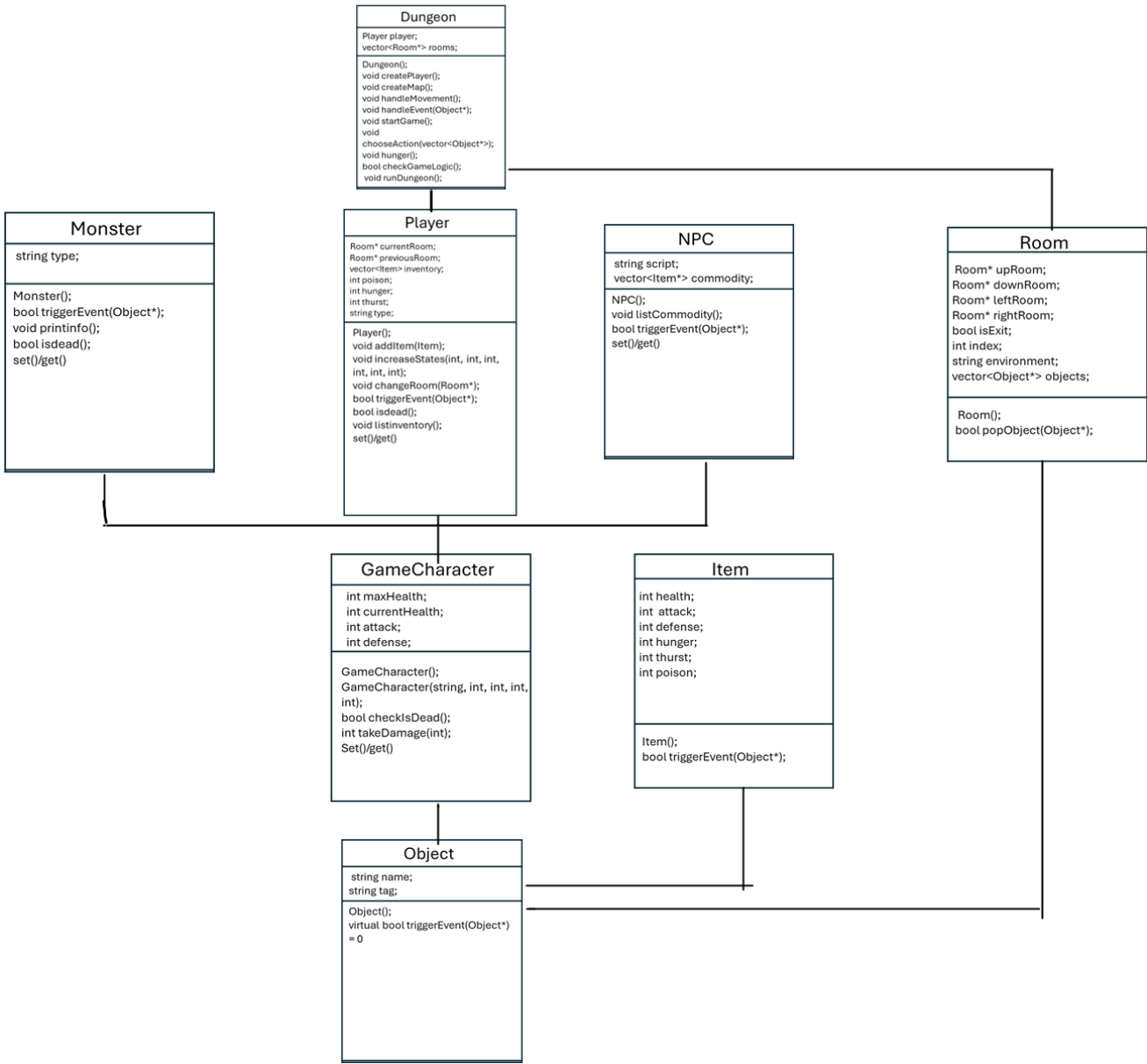


一、 UML 設計



二、 主要功能介紹

1. Movement

處理 movement 的函式(handlMovement)為 class Dungeon 的函式

```

void Dungeon::handleMovement()
{
    cout << "What's your next move?" << endl;

    if (player.getCurrentRoom()->getIsExit())
        cout << "Press [e] to exit." << endl;
    if (player.getCurrentRoom()->getRightRoom() != nullptr)
        cout << "Press [r] to go right" << endl;
    if (player.getCurrentRoom()->getLeftRoom() != nullptr)
        cout << "Press [l] to go left" << endl;
    if (player.getCurrentRoom()->getUpRoom() != nullptr)
        cout << "Press [u] to go up" << endl;
    if (player.getCurrentRoom()->getDownRoom() != nullptr)
        cout << "Press [d] to go down." << endl;
}

```

共五個選項，u(上)、d(下)、l(左)、r(右)、e(出去)，只有當前房間在該方向有連接房間才顯示該方向之選項；只有當前方向 isexit 為 true 才顯示出去的選項。

```

char choice;
cin >> choice;
if ((choice == 'e' || choice == 'E') && player.getCurrentRoom()->getIsExit())
{
    cout << "congratulations, you are the winner!!!" << endl;
    exit(0);
}
else if ((choice == 'R' || choice == 'r') && player.getCurrentRoom()->getRightRoom() != nullptr)
{
    player.changeRoom(player.getCurrentRoom()->getRightRoom());
}
else if ((choice == 'l' || choice == 'L') && player.getCurrentRoom()->getLeftRoom() != nullptr)
{
    player.changeRoom(player.getCurrentRoom()->getLeftRoom());
}
else if ((choice == 'u' || choice == 'U') && player.getCurrentRoom()->getUpRoom() != nullptr)
{
    player.changeRoom(player.getCurrentRoom()->getUpRoom());
}
else if ((choice == 'd' || choice == 'D') && player.getCurrentRoom()->getDownRoom() != nullptr)
{
    player.changeRoom(player.getCurrentRoom()->getDownRoom());
}

else
    cout << "Invalid operation. Please try again" << endl;
}

```

若按下 e，成功出地窖，輸出恭喜後結束遊戲。若按下四個方向，則用 class Player 的函式 changeroom 換到該方向的房間。

```

void Player::changeRoom(Room* newroom)
{
    previousRoom = currentRoom;
    currentRoom = newroom;
    cout << "The environment of this room is " << currentRoom->getenvironment() << endl;
}

```

changeroom 將 previousRoom 設為 currentRoom，再將要換過去的房間設為 currentRoom。並且印出換過去後的房間生態系為何。

2. Choose action

處理動作選擇的函式為 class Dungeon 中的 chooseAction

在第一次進入房間時，會先與怪獸打鬥接著與 NPC 互動，互動後清除 Room 裡存放 object 的 vector(接下來再次踏入同個房間將不會再有怪獸及 NPC)，再選擇接下來的動作。輸入 m 則選擇

接下來往哪邊走；輸入 s 輸出基本狀態(current health、offense、defense)，後面有詳細解釋；輸入 h 輸出 hunger system 數據(hunger、thurst、poison)，後面有詳細解釋；輸入 l 則列出身上道具，後面有詳細解釋；輸入 q 直接中止程式。

```
void Dungeon::chooseAction(vector<Object*> obj)
{
    for (auto i : obj)
    {
        handleEvent(i);
        player.getCurrentRoom()->popObject(i);
    }
    obj.clear();
    while (obj.size() > 0)
        obj.pop_back();
    cout << "What's you're next action?" << endl;
    char action;
    cout << "Press [m] to move" << endl;
    cout << "Press [s] to show status." << endl;
    cout << "Press [q] to quit." << endl;
    cout << "Press [h] to show hunger." << endl;
    cout << "Press [l] to list inventory." << endl;
    cin >> action;
    if (action == 'm' || action == 'M')
    {
        handleMovement();
    }
    else if (action == 's' || action == 'S')
    {
        player.triggerEvent(&player);
    }
    else if (action == 'q' || action == 'Q')
    {
        exit(0);
    }
    else if (action == 'h' || action == 'H') {
```

```
        cout << player.getName() << "'s hunger: " << player.gethunger() << ", thurst: " << player.getthurst() << ", poison: " << player.getpoison() << endl;
    }
    else if (action == 'l' || action == 'L') {
        player.listinventory();
    }
    else
        cout << "Invalid Operation. Please try again." << endl;
}
```

3. Showing status

我在我將 player 的資訊分成三部分，一個是基本資訊(current health, attack, defense)，一個是 hunger system 的資訊(hunger, thurst, poison)，一個是列出身上的道具。

印出基本資訊的函式為 class Player 中的 triggerEvent (class Object 中的 virtual function)

```
bool Player::triggerEvent(Object* obj)
{
    cout << getName() << "'s current health is: " << getCurrentHealth() << ", attack is: " << getAttack() << ", defense is: " << getDefense() << endl;
    return true;
}
```

出 hunger system 資訊的部分我沒有另外寫函式，直接在 class Dungeon 中的 chooseAction 函式(後面會再提到)實作。

```

else if (action == 'h' || action == 'H') {
    cout << player.getName() << "'s hunger: " << player.getHunger() << ", thirst: " << player.getThirst() << ", poison: " << player.getPoison() << endl;
}

```

列出道具的函式則為 class Player 中的 listinventory 函式。若 vector inventory 裡無東西，便印出 There is nothing in your backpack. 若有的話則用迴圈一件一件列出來。

```

void Player::listinventory() {
    if (inventory.size() == 0)
        cout << "There is nothing in your backpack." << endl;
    else
    {
        cout << "Items in your bag: ";
        for (auto i : inventory) {
            cout << i.getName() << "/";
        }
        cout << endl;
    }
}

```

4. NPC

在 player 遇到 NPC 時，NPC 列出物品名及屬性，輸入物品名後若為 nothing 直接 return，若為物品名稱，便靠迴圈比對，在 NPC 的 item 的 vector 找到後放入 player 的背包。

```

bool NPC::triggerEvent(Object* obj)
{
    Player* player = dynamic_cast<Player*>(obj);
    cout << this->getName() << ": " << this->script << endl;
    cout << "What do you want? You can only choose one thing.(Type [nothing] to get nothing.)" << endl;
    cout << "-----" << endl;
    this->listCommodity();
    cout << "-----" << endl;
    std::string thing;
    cin >> thing;
    if (thing == "nothing")
        return true;
    else
    {
        for (auto i : commodity)
        {
            if (i->getName() == thing)
            {
                i->triggerEvent(player);
                return true;
            }
        }
    }
    return false;
}

```

5. Pick up item

如前述，player 遇到 NPC 後，選擇物品並放入背包，放入背包的函式便是由 class item 中的 triggerEvent(class Object 中的 virtual function)與 class Player 中的 additem 負責。在 triggerEvent 中傳入的 Object 為 player，在 additem 中先將 item 放入 player 放 item 的 vector 中，再利用 increasestate 函式(class Player 中的函式)增加各項數據。接著回到 triggerEvent 中，印出 You've picked up (item 名)。

```

~bool Item::triggerEvent(Object* obj)
{
    Player* player = dynamic_cast<Player*>(obj);
    player->addItem(*this);
    cout << "You've picked up " << this->getName() << endl;
    return true;
}

```

```

~void Player::addItem(Item item)
{
    inventory.push_back(item);
    increaseStates(item.getHealth(), item.getAttack(), item.getDefense(), item.getHunger(), item.getThurst(), item.getPoison());
}

~void Player::increaseStates(int health, int attack, int defense, int Hunger, int Thurst, int Poison)
{
    setAttack(getAttack() + attack);
    setDefense(getDefense() + defense);
    setCurrentHealth(getCurrentHealth() + health);
    setHunger(getHunger() + Hunger);
    setThurst(getThurst() + Thurst);
    setPoison(getPoison() + Poison);
}

```

6. Fighting system and counter

在對打系統中，我加入了 **counter** 的元素，在創角時多了選擇種類的步驟，玩家可以選擇 **fire**、**water**、**grass** 三種元素，怪獸也加入這三種元素。

```

~void Dungeon::createPlayer()
{
    cout << "Please enter your name" << endl;
    string name;
    cin >> name;
    player = Player(name, 2000, 500, 500, 100, 100, 0);
    cout << "What type do you want to have? Fire, water, or grass?" << endl;
    string type;
    cin >> type;
    player.setType(type);
    cout << "Welcome to Dungeon, " << name << "!!!" << endl;
    player.triggerEvent(&player);
}

```

```

Monster* ghidorah = new Monster("Ghidorah", 1200, 600, 600, "fire");
Monster* godzilla = new Monster("Godzilla", 1000, 500, 500, "water");
Monster* kong = new Monster("Kong", 1000, 500, 500, "grass");
Monster* mothra = new Monster("Mothra", 800, 400, 400, "fire");
Monster* rodan = new Monster("Rodan", 700, 350, 350, "water");
Monster* behemoth = new Monster("Behemoth", 700, 350, 350, "grass");
Monster* scylla = new Monster("Scylla", 600, 300, 300, "fire");
Monster* methuselah = new Monster("Methuselah", 600, 300, 300, "water");
Monster* muto = new Monster("MUTO", 600, 300, 300, "grass");

```

對打系統由 `class Monster` 中的 `triggerEvent` (`class Object` 的 `virtual function`)負責，將 `player` 傳入 `triggerEvent` 中，先印出怪獸的屬性(`printf`)後選擇對打或撤退。

```

~void Monster::printf()
{
    cout << getName()<<"'s type is: "<<type << ", current health is: " << getCurrentHealth() << ", attack is: " << getAttack() << ", defense is: " << getDefense()
}

```

```

12  bool Monster::triggerEvent(Object* obj)
13  {
14      Player* player = dynamic_cast<Player*>(obj);
15      cout << "You've encountered a monster, " << getName() << "." << endl;
16      printinfo();
17      player->triggerEvent(player);
18      cout << "what do you want to do? Attack or retreat?" << endl;
19      std::string choice;
20      cin >> choice;
21      if (choice == "attack")
22      {
23          cout << "Fight!!!" << endl;

```

若選擇攻打，則會進入迴圈內直到某一方死亡。首先會遇到 counter 判斷(counter 順序為 fire->grass->water->fire)，我設定了兩個係數分別屬於怪獸及玩家，若玩家康特怪獸，則玩家係數為 1.2，怪獸係數為 0.8；反之怪獸康特玩家則玩家係數為 0.8，怪獸係數為 1.2；若兩者屬性同則細數皆為 1，係數會在計算傷害時乘上照公式算出的傷害以作為最終傷害(下圖程式碼過長僅顯示各一種屬性的判斷式，其他項目以此類推)。

```

while (this->getCurrentHealth() > 0 && player->getCurrentHealth() > 0)
{
    float p = 1, m = 1;
    if (player->gettype() == "grass") {
        if (type == "fire")
        {
            p = 0.8;
            cout << "You are countered." << endl;
        }
        else if (type == "grass")
            p = 1;
        else if (type == "water")
        {
            p = 1.2;
            cout << "You counter the monster." << endl;
        }
    }
    else if (player->gettype() == "fire") { ... }
    else if (player->gettype() == "water") { ... }
}

```

```

1  if (type == "grass") {
2      if (player->gettype() == "fire")
3          m = 0.8;
4      else if (player->gettype() == "grass")
5          m = 1;
6      else if (player->gettype() == "water")
7          m = 1.2;
8  }
9  else if (type == "fire") { ... }
7  else if (type == "water") { ... }
5

```

傷害公式為:攻擊者康特係數(如上述)*隨機 1~4 之間的數*(攻擊者攻擊*攻擊者攻擊/防守者防守)/5。在任一方攻擊後皆會顯示被攻擊方資訊，玩家攻擊後判斷怪物是否死亡，若死亡則輸出玩家殺死(怪獸名)，並且獲得些許恢復及提升—current health, attack, defense 各乘以 1.05。若怪獸未死，則換怪獸攻擊，且怪獸攻擊時，有 1/5 的機率受到毒擊，中毒(poison)+1。若玩家死

亡則輸出遊戲結束，反之兩者皆為死亡則重複迴圈直至一方死亡。

```
int dmg = p*((rand() % 4) + 1) * (player->getAttack() * player->getAttack() / this->getDefense()) / 5;
cout << "You attacked monster, causing " << dmg << " damages" << endl;
if (dmg <= getCurrentHealth())
    this->setCurrentHealth(takeDamage(dmg));
else
    this->setCurrentHealth(0);
this->printinfo();

if (this->isdead())
{
    cout << "You killed " << this->getName() << ". You got recovery." << endl;
    player->setAttack(player->getAttack() * 1.05);
    if (player->getCurrentHealth() * 1.05 <= player->getMaxHealth())
    {
        player->setCurrentHealth(player->getCurrentHealth() * 1.05);
    }
    else
        player->setCurrentHealth(player->getMaxHealth());
    player->setDefense(player->getDefense() * 1.05);
    player->triggerEvent(player);
    system("pause");
    return true;
}
```

```
else
{
    dmg = m*((rand() % 4) + 1) * (this->getAttack() * this->getAttack() / player->getDefense())/5;
    cout << "You was attacked by monster, causing " << dmg << " damages" << endl;
    if (rand() % 5 == 0) {
        cout << this->getName() << " spits venom. You are poisoned." << endl;
        cout << "state increases: poison + 1" << endl;
        player->setpoison(player->getpoison() + 1);
    }
    if (dmg <= player->getCurrentHealth())
        player->setCurrentHealth(player->takeDamage(dmg));
    else
        player->setCurrentHealth(0);
    player->triggerEvent(player);
}

if (player->isdead())
{
    cout << "You are dead. Game over....." << endl;
    exit(0);
    return false;
}

return false;
```

若在一開始未選擇 attack 而是選擇 retreat，則用 changeRoom 函式退回上個房間，並且 current health, attack, defense 三項數據接受到降低，皆乘以 0.8，

```

else if (choice == "retreat")
{
    player->changeRoom(player->getPreviousRoom());
    cout << " You are such a loser! You got debuffed!" << endl;
    player->setAttack(player->getAttack() * 0.8);
    player->setCurrentHealth(player->getCurrentHealth() * 0.8);
    player->setDefense(player->getDefense() * 0.8);
    player->triggerEvent(player);
    return true;
}
return false;

```

7. Game logic and runDungeon

class `Dungeon` 中 `GameLogic` 函式 判斷遊戲是否在正確狀態，若玩家血量小於等於 0 或是處在的房間不存在，則回傳 `false`。

```

bool Dungeon::checkGameLogic()
{
    if (player.getCurrentRoom() == nullptr || player.getCurrentHealth() <= 0)
        return false;
    else
        return true;
}

```

class `Dungeon` 中 `runDungeon` 函式控制整個系統的運作，若 `checkGameLogic` 為否便終止程式。此函式中包含選擇下一個動作(`chooseAction`)以及在每一輪選擇後 `hunger` 數據減少，另外我在選擇前添加了受傷的事件，發生機率為 1/6，若發生了則 `current health` 減少 100。

```

void Dungeon::runDungeon() {
    startGame();
    while (checkGameLogic())
    {
        if (rand() % 6 == 0) {
            cout << "You are injured accidently" << endl;
            cout << "Status decreases: current health - 100" << endl;
            player.setCurrentHealth(player.getCurrentHealth() - 100);
        }
        chooseAction(player.getCurrentRoom()->getObjects());
        hunger();
        system("pause");
    }
    exit(0);
}

```

8. Hunger system

Hunger system 負責在每一輪的選擇動作後進行例行的 `hunger` 數據消耗或是中毒數據增加。Hunger 與 `thurst` 消耗程度與 `poison` 程度呈正相關，舉例來說在沙漠中，每一輪的選擇皆會消耗 $3 * (\text{poison} + 1)$ 的 `hunger` 與 $6 * \text{thurst} * (\text{poison} + 1)$ ，若遇到沙塵暴(發生機率 1/2)則數據加倍，另外也有 2/5 機率遇到綠洲，則 `hunger+6`、`thurst+12`。若任何一項數據歸零，則改減少 `current health`。

```

void Dungeon::hunger() {
    int i;
    int j = rand() % 5;
    if (rand() % 2 == 1)
        i = 2;
    else
        i = 1;
    if (player.getCurrentRoom()->getenvironment() == "Desert") {
        if (i == 2)
            cout << "You encountered sand storm, thirst and hunger decrease dramatically" << endl;
            if (player.getthunger() > 0) {
                player.setthunger(player.getthunger() - i * 3 * (player.getpoison() + 1));
                cout << "state decreases: hunger - " << i * 3 * (player.getpoison() + 1) << endl;
            }
        else
        {
            player.setCurrentHealth(player.getCurrentHealth() - i * 3 * (player.getpoison() + 1));
            cout << "state decreases: current health - " << i * 3 * (player.getpoison() + 1) << endl;
        }
        if (player.getthirst() > 0) {
            player.setthirst(player.getthirst() - i * 6 * (player.getpoison() + 1));
            cout << "state decreases: thirst - " << i * 6 * (player.getpoison() + 1) << endl;
        }
    }
    else
    {
        player.setCurrentHealth(player.getCurrentHealth() - i * 6 * (player.getpoison() + 1));
        cout << "state decreases: current health - " << i * 6 * (player.getpoison() + 1) << endl;
    }
    if (j == 2 || j == 4) {
        cout << "You find an oasis" << endl;
        cout << "state increases: hunger + 6" << endl << "state increases: thirst + 12" << endl;
        player.setthunger(player.getthunger() + 6);
        player.setthirst(player.getthirst() + 12);
    }
}

```

在森林中則是每一輪的選擇皆會消耗 $6 * (\text{poison} + 1)$ 的 hunger 與 $3 * \text{thirst} * (\text{poison} + 1)$ ，若遇到沙塵暴(發生機率 1/2)則數據加倍，另外也有 2/5 機率遇到湖泊，則 hunger+12、thirst+6。若任何一項數據歸零，則改減少 current health。

```

if (player.getCurrentRoom()->getenvironment() == "Forest") {
    if (i == 2)
        cout << "You encountered wild fire, thirst and hunger decrease dramatically" << endl;
    if (player.getthunger() > 0) {
        player.setthunger(player.getthunger() - i * 6 * (player.getpoison() + 1));
        cout << "state decreases: hunger - " << i * 6 * (player.getpoison() + 1) << endl;
    }
    else
    {
        player.setCurrentHealth(player.getCurrentHealth() - i * 6 * (player.getpoison() + 1));
        cout << "state decreases: current health - " << i * 6 * (player.getpoison() + 1) << endl;
    }
    if (player.getthirst() > 0) {
        player.setthirst(player.getthirst() - i * 3 * (player.getpoison() + 1));
        cout << "state decreases: thirst - " << i * 3 * (player.getpoison() + 1) << endl;
    }
    else
    {
        player.setCurrentHealth(player.getCurrentHealth() - i * 3 * (player.getpoison() + 1));
        cout << "state decreases: current health - " << i * 3 * (player.getpoison() + 1) << endl;
    }
    if (j == 2 || j == 4) {
        cout << "You find a lake" << endl;
        cout << "state increases: hunger + 12" << endl << "state increases: thirst + 6" << endl;
        player.setthunger(player.getthunger() + 12);
        player.setthirst(player.getthirst() + 6);
    }
}
}

```

在沼澤中則是每一輪的選擇皆會消耗 $3 \times (\text{poison} + 1)$ 的 hunger 與 $3 \times \text{thurst} \times (\text{poison} + 1)$ ，且會增加中毒程度 1，若遇到神群出沒 (發生機率 1/2) 則中毒程度+2，另外也有 2/5 機率遇到牛奶，則 $\text{poison} - 2$ 。若任何一項數據歸零，則改減少 current health。

```
if (player.getCurrentRoom()->getenvironment() == "Swamp") {
    if (i == 2)
    {
        cout << "Snakes come in great amount, you are poisoned by snakes venom seriously." << endl;
        cout << "state increases: poison + 2" << endl;
        player.setpoison(player.getpoison() + 2);
    }
    else {
        cout << "You are poisoned by sake venom" << endl;
        cout << "state increases: poison + 1" << endl;
        player.setpoison(player.getpoison() + 1);
    }
}

if (player.getthunger() > 0) {
    player.sethunger(player.getthunger() - i * 3 * (player.getpoison() + 1));
    cout << "state decreases: hunger - " << i * 3 * (player.getpoison() + 1) << endl;
}
else
{
    player.setCurrentHealth(player.getCurrentHealth() - i * 3 * (player.getpoison() + 1));
    cout << "state decreases: current health - " << i * 3 * (player.getpoison() + 1) << endl;
}

if (player.getthurst() > 0) {
    player.setthurst(player.getthurst() - i * 3 * (player.getpoison() + 1));
    cout << "state decreases: thurst - " << i * 3 * (player.getpoison() + 1) << endl;
}
else
{
    player.setCurrentHealth(player.getCurrentHealth() - i * 3 * (player.getpoison() + 1));
    cout << "state decreases: current health - " << i * 3 * (player.getpoison() + 1) << endl;
}

if (j == 2 || j == 4) {
    cout << "You find a bottle of milk" << endl;
    cout << "state decreases: poison - 2" << endl;
    if ((player.getpoison() - 2) > 0)
```

9. Room system

關於生態系所帶來的數據減少已設計於 hunger system，這裡只說明房間創建
創建怪獸

```
Monster* ghidorah = new Monster("Ghidorah", 1200, 600, 600, "fire");
Monster* godzilla = new Monster("Godzilla", 1000, 500, 500, "water");
Monster* kong = new Monster("Kong", 1000, 500, 500, "grass");
Monster* mothra = new Monster("Mothra", 800, 400, 400, "fire");
Monster* rodan = new Monster("Rodan", 700, 350, 350, "water");
Monster* behemoth = new Monster("Beheoth", 700, 350, 350, "grass");
Monster* scylla = new Monster("Scylla", 600, 300, 300, "fire");
Monster* methuselah = new Monster("Methuselah", 600, 300, 300, "water");
Monster* muto = new Monster("MUTO", 600, 300, 300, "grass");
```

創建 NPC

```
NPC* Dr_Mark = new NPC("Dr. Mark Russell", "Monster can't live with people, we need to kill them all. Let me help you!!!", obj1);
NPC* Dr_Emma = new NPC("Dr. Emma Russell", "You need to be careful with Ghidorah. He is the king of monster. Let me give you something to become stronger.", obj2);
NPC* Madison = new NPC("Madison", "You look so tired, how can I help you?", obj3);
NPC* Dr_Rick = new NPC("Dr. Rick Stanton", "The remaining half is more difficult. You need more weapons ", obj4);
NPC* Dr_Sam = new NPC("Dr. Sam Coleman", "We believe you can save the people. Let me help you.", obj5);
NPC* Dr_Vivienne = new NPC("Dr. Vivienne Graham", "Muto is strong to beat. You must need some supply", obj6);
NPC* Dr_Ishiro = new NPC("Dr. Ichiro Serizawa", "You almost arrive the exit. Let me give you a helping hand", obj7);
```

創建補給品以及 NPC 所攜帶補給品之 vector

```
Item* protein = new Item("protein", 100, 0, 0, 0, 0, 0);
Item* creatine = new Item("creatine", 150, 0, 0, 0, 0, 0);
Item* starch = new Item("starch", 175, 0, 0, 0, 0, 0);
Item* fat = new Item("fat", 200, 0, 0, 0, 0, 0);
Item* water = new Item("water", 0, 0, 0, 0, 10, 0);
Item* juice = new Item("juice", 0, 0, 0, 0, 15, 0);
Item* coke = new Item("coke", 0, 0, 0, 0, 20, 0);
Item* pistol = new Item("pistol", 0, 50, 0, 0, 0, 0);
Item* shotgun = new Item("shotgun", 0, 55, 0, 0, 0, 0);
Item* submachine_gun = new Item("submachineGun", 0, 60, 0, 0, 0, 0);
Item* rifle = new Item("rifle", 0, 65, 0, 0, 0, 0);
Item* sniper_riflen = new Item("sniperRifle", 0, 70, 0, 0, 0, 0);
Item* machine_gun = new Item("machineGun", 0, 75, 0, 0, 0, 0);
Item* milk = new Item("milk", 0, 0, 0, 0, 0, -3);
Item* yogurt = new Item("yogurt", 0, 0, 0, 0, 0, -3);
Item* ice_cream = new Item("iceCream", 0, 0, 0, 0, 0, -4);
Item* sheild = new Item("sheild", 0, 0, 50, 0, 0, 0);
Item* bulletproof_vest = new Item("bulletproofVest", 0, 0, 100, 0, 0, 0);
Item* pizza = new Item("pizza", 0, 0, 0, 10, 0, 0);
Item* sandwich = new Item("sandwich", 0, 0, 0, 15, 0, 0);
Item* donut = new Item("donut", 0, 0, 0, 20, 0, 0);
vector<Item*>obj1{ shotgun,protein,pizza };
vector<Item*>obj2{ submachine_gun,water,bulletproof_vest };
vector<Item*>obj3{ rifle,creatine,ice_cream };
vector<Item*>obj4{ machine_gun,juice,sandwich };
vector<Item*>obj5{ sniper_riflen,starch,yogurt };
vector<Item*>obj6{ pistol,coke,milk };
vector<Item*>obj7{ sheild,fat,donut };
```

創建房間:設定是否為出口、生態系、放入角色(怪獸與 NPC)

```
Room* Monarch0 = new Room(false, 0, " ", vector<Object*>{});
Room* Monarch1 = new Room(false, 1, "Desert",vector<Object*>{Dr_Mark});
Room* Monarch2 = new Room(false, 2, "Forest",vector<Object*>{godzilla, Dr_Emma});
Room* Monarch3 = new Room(false, 3, "Swamp",vector<Object*>{methuselah});
Room* Monarch4 = new Room(false, 4, "Desert", vector<Object*>{kong});
Room* Monarch5 = new Room(false, 5, "Forest", vector<Object*>{mothra});
Room* Monarch6 = new Room(false, 6, "Swamp", vector<Object*>{rodan});
Room* Monarch7 = new Room(false, 7, "Desert", vector<Object*>{Madison});
Room* Monarch8 = new Room(false, 8, "Forest", vector<Object*>{behemoth});
Room* Monarch9 = new Room(false, 9, "Swamp", vector<Object*>{Dr_Sam});
Room* Monarch10 = new Room(false, 10, "Desert", vector<Object*>{scylla});
Room* Monarch11 = new Room(false, 11, "Forest", vector<Object*>{Dr_Rick});
Room* Monarch12 = new Room(false, 12, "Swamp", vector<Object*>{Dr_Vivienne});
Room* Monarch13 = new Room(false, 13, "Desert", vector<Object*>{muto });
Room* Monarch14 = new Room(false, 14, "Forest", vector<Object*>{ Dr_Ishiro});
Room* Monarch15 = new Room(true, 15, "Swamp", vector<Object*>{ghidorah});
```

將房間放入 rooms 的 vector

```
rooms.push_back(Monarch0);
rooms.push_back(Monarch1);
rooms.push_back(Monarch2);
rooms.push_back(Monarch3);
rooms.push_back(Monarch4);
rooms.push_back(Monarch5);
rooms.push_back(Monarch6);
rooms.push_back(Monarch7);
rooms.push_back(Monarch8);
rooms.push_back(Monarch9);
rooms.push_back(Monarch10);
rooms.push_back(Monarch11);
rooms.push_back(Monarch12);
rooms.push_back(Monarch13);
rooms.push_back(Monarch14);
rooms.push_back(Monarch15);
```

設定 **player** 的初始房間，並將房間串起來。

```
player.setCurrentRoom(Monarch0);
Monarch0->setRightRoom(Monarch1);
Monarch0->setUpRoom(Monarch9);
Monarch1->setUpRoom(Monarch2);
Monarch1->setDownRoom(Monarch3);
Monarch1->setRightRoom(Monarch5);
Monarch1->setLeftRoom(Monarch0);
Monarch2->setUpRoom(Monarch6);
Monarch2->setDownRoom(Monarch3);
Monarch2->setRightRoom(Monarch10);
Monarch3->setUpRoom(Monarch1);
Monarch3->setRightRoom(Monarch4);
Monarch4->setUpRoom(Monarch7);
Monarch4->setLeftRoom(Monarch3);
Monarch4->setRightRoom(Monarch14);
Monarch5->setLeftRoom(Monarch1);
Monarch5->setRightRoom(Monarch7);
Monarch6->setDownRoom(Monarch2);
Monarch6->setLeftRoom(Monarch9);
Monarch7->setUpRoom(Monarch11);
Monarch7->setDownRoom(Monarch4);
Monarch7->setLeftRoom(Monarch5);
Monarch7->setRightRoom(Monarch8);
Monarch8->setRightRoom(Monarch13);
Monarch8->setLeftRoom(Monarch7);
Monarch9->setDownRoom(Monarch0);
Monarch9->setRightRoom(Monarch6);
Monarch10->setRightRoom(Monarch11);
Monarch10->setLeftRoom(Monarch2);
Monarch11->setLeftRoom(Monarch10);
Monarch11->setRightRoom(Monarch12);
Monarch12->setDownRoom(Monarch15);
Monarch12->setLeftRoom(Monarch11);
Monarch13->setDownRoom(Monarch14);
```

```
Monarch13->setDownRoom(Monarch14);
Monarch13->setLeftRoom(Monarch8);
Monarch14->setUpRoom(Monarch13);
Monarch14->setLeftRoom(Monarch4);
Monarch14->setRightRoom(Monarch15);
Monarch15->setUpRoom(Monarch12);
Monarch15->setLeftRoom(Monarch14);
```

三、 成果:

於資料夾中附上遊戲輸出檔案 **game over.txt** 與 **you win.txt**

在此僅詳細說明 **game over.txt** 中的過程

輸入名稱與類別(counter 系統)

```
Please enter your name
Richard
What type do you want to have? Fire, water, or grass?
water
Welcome to Dungeon, Richard!!!
Richard's current health is: 2000, attack is: 500, defense is: 500
```

選擇接下來的動作，按下 **m** 顯示可以奏的方向，選擇向右走後顯示房間生態系以及 **hunger system** 數據增減

```
What's your next action?
Press [m] to move
Press [s] to show status.
Press [q] to quit.
Press [h] to show hunger.
Press [l] to list inventory.
m
What's your next move?
Press [r] to go right
Press [u] to go up
r
The environment of this room is Desert
state decreases: hunger - 3
state decreases: thirst - 6
You find an oasis
state increases: hunger + 6
state increases: thirst + 12
請按任意鍵繼續 . . .
```

遇到 **NPC**，輸出台詞、商品，並選擇商品

```
請按任意鍵繼續 . . .
Dr. Mark Russell: Monster can't live with people, we need to kill them all. Let me help you!!!
What do you want? You can only choose one thing.(Type [nothing] to get nothing.)
-----
shotgun: health: 0, attack: 55, defense: 0, hunger: 0, thirst: 0, poison 0
protein: health: 100, attack: 0, defense: 0, hunger: 0, thirst: 0, poison 0
pizza: health: 0, attack: 0, defense: 0, hunger: 10, thirst: 0, poison 0
-----
pizza
You've picked up pizza
```

選擇 **l** 後列出背包物品，並且 **hunger system** 數據增減

```
What's your next action?
Press [m] to move
Press [s] to show status.
Press [q] to quit.
Press [h] to show hunger.
Press [l] to list inventory.
l
Items in your bag: pizza/
state decreases: hunger - 3
state decreases: thirst - 6
You find an oasis
state increases: hunger + 6
state increases: thirst + 12
請按任意鍵繼續 . . .
```

遇到隨機出現的受傷事件，健康-100

```
You are injured accidentally
Status decreases: current health - 100
```

選擇 **s** 後輸出基本數據，及 **hunger system** 數據增減

```
Press [m] to move
Press [s] to show status.
Press [q] to quit.
Press [h] to show hunger.
Press [l] to list inventory.

s
Richard's current health is: 1900, attack is: 500, defense is: 500
state decreases: hunger - 3
state decreases: thirst - 6
請按任意鍵繼續 . . .
```

選擇 **h** 後輸出 **hunger system** 數據，且遇上沙塵暴事件，**hunger system** 數據倍減

```
What's you're next action?
Press [m] to move
Press [s] to show status.
Press [q] to quit.
Press [h] to show hunger.
Press [l] to list inventory.

h
Richard's hunger: 113, thirst: 106, poison: 0
You encountered sand storm, thirst and hunger decrease dramatically
state decreases: hunger - 6
state decreases: thirst - 12
請按任意鍵繼續 . . .
```

選擇 **m** 後輸出可移動方向，選擇往下輸出新房間環境，且遇上蛇群出沒，中毒+2，但撿到牛奶中毒-2，**hunger system** 數據增減。

```
m
What's your next move?
Press [r] to go right
Press [l] to go left
Press [u] to go up
Press [d] to go down.

d
The environment of this room is Swamp
Snakes come in great amount, you are poisoned by snakes venom seriously.
state increases: poison + 2
state decreases: hunger - 18
state decreases: thirst - 18
You find a bottle of milk
state decreases: poison - 2
請按任意鍵繼續 . . .
```

遇上怪獸，選擇撤退，基本數據受到 **debuff**。

```
You've encountered a monster, Methuselah.
Methuselah's type is: water, current health is: 600, attack is: 300, defense is: 300
Richard's current health is: 1900, attack is: 500, defense is: 500
what do you want to do? Attack or retreat?
retreat
The environment of this room is Desert
You are such a loser! You got debuffed!
Richard's current health is: 1520, attack is: 400, defense is: 400
```

選擇往下走，**hunger system** 數據增減。

```

Richard's current health is: 1520, attack is: 400, defense is: 400
What's your next action?
Press [m] to move
Press [s] to show status.
Press [q] to quit.
Press [h] to show hunger.
Press [l] to list inventory.
m
What's your next move?
Press [r] to go right
Press [l] to go left
Press [u] to go up
Press [d] to go down.
r
The environment of this room is Forest
state decreases: hunger - 12
state decreases: thirst - 6
請按任意鍵繼續 . . .

```

遇到怪獸選擇攻打並且獲勝，數據獲得些許恢復。

```

You've encountered a monster, Mothra.
Mothra's type is: fire, current health is: 800, attack is: 400, defense is: 400
Richard's current health is: 1520, attack is: 400, defense is: 400
what do you want to do? Attack or retreat?
attack
Fight!!!
You counter the monster.
You attacked monster, causing 96 damages
Mothra's type is: fire, current health is: 704, attack is: 400, defense is: 400
You was attacked by monster, causing 256 damages
Richard's current health is: 1264, attack is: 400, defense is: 400
You counter the monster.
You attacked monster, causing 288 damages
Mothra's type is: fire, current health is: 416, attack is: 400, defense is: 400
You was attacked by monster, causing 128 damages
Richard's current health is: 1136, attack is: 400, defense is: 400
You counter the monster.
You attacked monster, causing 288 damages
Mothra's type is: fire, current health is: 128, attack is: 400, defense is: 400
You was attacked by monster, causing 128 damages
Richard's current health is: 1008, attack is: 400, defense is: 400
You counter the monster.
You attacked monster, causing 288 damages
Mothra's type is: fire, current health is: 0, attack is: 400, defense is: 400
You killed Mothra. You got recovery.
Richard's current health is: 1058, attack is: 420, defense is: 420
請按任意鍵繼續 . . .

```

選擇往下走，hungersystem 數據增減。

```

Richard's current health is: 1058, attack is: 420, defense is: 420
請按任意鍵繼續 . . .
What's your next action?
Press [m] to move
Press [s] to show status.
Press [q] to quit.
Press [h] to show hunger.
Press [l] to list inventory.
m
What's your next move?
Press [r] to go right
Press [l] to go left
r
The environment of this room is Desert
You encountered sand storm, thirst and hunger decrease dramatically
state decreases: hunger - 12
state decreases: thirst - 24
請按任意鍵繼續 . . .

```

意外受傷健康-100，並且遇到 NPC，選取道具。

```

You are injured accidentally
Status decreases: current health - 100
Madison: You look so tired, how can I help you?
Waht do you want? You can only choose one thing.(Type [nothing] to get nothing.)
-----
rifle: health: 0, attack: 65, defense: 0,hunger: 0,thurst: 0,poison 0
creatine: health: 150, attack: 0, defense: 0,hunger: 0,thurst: 0,poison 0
iceCream: health: 0, attack: 0, defense: 0,hunger: 0,thurst: 0,poison -4
-----
rifle
You've picked up rifle

```

選擇 s，列出基本數據，hunger syste, 數據增減。

```

What's you're next action?
Press [m] to move
Press [s] to show status.
Press [q] to quit.
Press [h] to show hunger.
Press [l] to list inventory.
s
Richard's current health is: 958, attack is: 485, defense is: 420
state decreases: hunger - 6
state decreases: thurst - 12
請按任意鍵繼續 . . .

```

選擇往下走，hUnger system 數據增減

```

What's you're next action?
Press [m] to move
Press [s] to show status.
Press [q] to quit.
Press [h] to show hunger.
Press [l] to list inventory.
m
What's your next move?
Press [r] to go right
Press [l] tp go left
Press [u] to go up
Press [d] to go down.
d
The environment of this room is Desert
You encoutered sand storm, thurst and hunger decrease dramatically
state decreases: hunger - 12
state decreases: thurst - 24
You find an oasis
state increases: hunger + 6
state increases: thurst + 12
請按任意鍵繼續 . . .

```

遇見怪獸，選擇攻擊後遭到打敗，遊戲結束

```

You've encountered a monster, Kong.
Kong's type is: grass, current health is: 1000, attack is: 500, defense is: 500
Richard's current health is: 958, attack is: 485, defense is: 420
what do you want to do? Attack or retreat?
attack
Fight!!!
You are countered.
You attacked monster, causing 300damages
Kong's type is: grass, current health is: 700, attack is: 500, defense is: 500
You was attacked by monster, causing 428 damages
Richard's current health is: 530, attack is: 485, defense is: 420
You are countered.
You attacked monster, causing 300damages
Kong's type is: grass, current health is: 400, attack is: 500, defense is: 500
You was attacked by monster, causing 428 damages
Richard's current health is: 102, attack is: 485, defense is: 420
You are countered.
You attacked monster, causing 150damages
Kong's type is: grass, current health is: 250, attack is: 500, defense is: 500
You was attacked by monster, causing 142 damages
Richard's current health is: 0, attack is: 485, defense is: 420
You are dead. Game over.....

```

四、 Discussion:

在這份作業當中，創地圖的部分原本是想做出能隨機生成的地圖，但在時間壓力和能力不

足的情況下，並未能實施，另外還有 **Record** 也一樣無暇去研究與實施，因此將來有機會的話會花時間去研究與實作。此外在一些系統如 **counter** 與 **hunger** 的設計上我認為並沒有寫得很好，尤其是 **counter** 系統直接寫在 **monster** 的 **triggerEvent**，會再思考如何寫得更好。

五、 **Conclusion:**

這是大概是我進到大學以來寫的第一個較複雜的作業，花上蠻多時間與精力在這個作業上，雖然因為是第一次所以很多地方寫得不夠好，且寫起來有點吃力，但完成之後很有成就感，而且也對 **class**、**inheritance**、**virtual function** 有更多的了解與實作。